

使用cando.dll二次开发 (只适用于Windos)

使用cando.dll二次开发 (只适用于Windos)

1. 内部变量和结构体
 - 1.1 CAN 工作模式标志位
 - 1.2 CAN ID 标志位
 - 1.3 CAN 总线错误标志位
 - 1.4 cando_frame_t 数据帧结构体
 - 1.5 cando_bittiming_t CAN波特率配置结构体
2. 接口说明
 - 2.1 设备列表相关接口
 - 2.1.1 cando_list_malloc
 - 2.1.2 cando_list_free
 - 2.1.3 cando_list_scan
 - 2.1.4 cando_list_num
 - 2.2 设备操作相关接口
 - 2.2.1 cando_malloc
 - 2.2.2 cando_free
 - 2.2.3 cando_open
 - 2.2.4 cando_close
 - 2.2.5 cando_get_serial_number_str
 - 2.2.6 cando_get_dev_info
 - 2.2.7 cando_set_timing
 - 2.2.8 cando_start
 - 2.2.9 cando_stop
 - 2.2.10 cando_frame_send
 - 2.2.11 cando_frame_read
 - 2.3 辅助功能接口
 - 2.3.1 cando_parse_err_frame

1. 内部变量和结构体

1.1 CAN 工作模式标志位

- CANDO_MODE_NORMAL 正常工作模式
- CANDO_MODE_LISTEN_ONLY CAN 侦听模式
- CANDO_MODE_LOOP_BACK CAN 回环模式
- CANDO_MODE_ONE_SHOT CAN 发送失败后不自动重新发送模式
- CANDO_MODE_NO_ECHO_BACK CAN 发送数据帧后不向电脑返回echo帧 (默认为返回echo帧)

1.2 CAN ID 标志位

- CANDO_ID_MASK 用于和 Frame.can_id 按位与运算, 得到 can id
- CANDO_ID_EXTENDED 用于和 Frame.can_id 按位与运算, 判断是否为扩展帧
- CANDO_ID_RTR 用于和 Frame.can_id 按位与运算, 判断是否为远程帧
- CANDO_ID_ERR 用于和 Frame.can_id 按位与运算, 判断是否错误帧

1.3 CAN 总线错误标志位

CAN_ERR_BUSOFF 离线错误
CAN_ERR_RX_TX_WARNING 发送或接收错误报警
CAN_ERR_RX_TX_PASSIVE 发送或接收被动错误
CAN_ERR_OVERLOAD 总线过载
CAN_ERR_STUFF 填充规则错误
CAN_ERR_FORM 格式错误
CAN_ERR_ACK 应答错误
CAN_ERR_BIT_RECESSIVE 位隐性错误
CAN_ERR_BIT_DOMINANT 位显性错误
CAN_ERR_CRC CRC校验错误

1.4 cando_frame_t 数据帧结构体

```
uint32_t echo_id    // 判断是否为发送的 ECHO 帧，ECHO 帧时值为 0，否则为 0xFFFFFFFF
uint32_t can_id     // 帧ID，用于判断帧类型，和`CANDO_ID_MASK`按位`与`运算得到can_id
uint8_t can_dlc     // can数据长度，0~8
uint8_t channel     // can通道，0 或 1
uint8_t flags;      // 用于内部通信，用户无需理会
uint8_t reserved;   // 暂未使用，用户无需理会
uint8_t data[8];    // can数据
uint32_t timestamp_us; // can 数据时间戳，单位为 us
```

1.5 cando_bittiming_t CAN波特率配置结构体

```
uint32_t prop_seg    // propagation Segment (固定为 1)
uint32_t phase_seg1  // phase segment 1 (1~15)
uint32_t phase_seg2  // phase segment 2 (1~8)
uint32_t sjw         // synchronization segment (1~4)
uint32_t brp         // CAN时钟分频 (1~1024)，内部CAN时钟为 48MHz
```

如果您不需要自己设定特殊的can波特率或采样点请参考 cando_example 源码中的常见波特率配置表。

2. 接口说明

2.1 设备列表相关接口

2.1.1 cando_list_malloc

```
bool cando_list_malloc(cando_list_handle *list)
```

- **函数说明：**创建设备列表
- ***list：**设备列表句柄指针，类型为 cando_list_handle *
- **返回值：**True，成功；False，失败

2.1.2 cando_list_free

```
bool cando_list_free(cando_list_handle list)
```

- **函数说明：**释放设备列表
- **list：**设备列表句柄，类型为 cando_list_handle

- **返回值:** True, 成功; False, 失败

2.1.3 cando_list_scan

```
bool cando_list_scan(cando_list_handle list)
```

- **函数说明:** 扫描当前电脑连接的所有Cando或Cando_pro设备
- **list:** 设备列表句柄, 类型为 cando_list_handle
- **返回值:** True, 成功; False, 失败

2.1.4 cando_list_num

```
bool cando_list_num(cando_list_handle list, uint8_t *num)
```

- **函数说明:** 获取扫描到的Cando或Cando_pro设备的数量
- **list:** 设备列表句柄, 类型为 cando_list_handle
- ***num:** 存储设备数量的变量指针
- **返回值:** True, 成功; False, 失败

2.2 设备操作相关接口

2.2.1 cando_malloc

```
bool cando_malloc(cando_list_handle list, uint8_t index, cando_handle *hdev)
```

- **函数说明:** 创建设备实例
- **list:** 设备列表句柄, 类型为 cando_list_handle
- **index:** 想要创建设备位于设备列表中的索引值, 从 0 开始
- ***hdev:** 设备句柄指针, 类型为 cando_handle *
- **返回值:** True, 成功; False, 失败

2.2.2 cando_free

```
bool cando_free(cando_handle hdev)
```

- **函数说明:** 释放设备实例
- **hdev:** 设备句柄, 类型为 cando_handle
- **返回值:** True, 成功; False, 失败

2.2.3 cando_open

```
bool cando_open(cando_handle hdev)
```

- **函数说明:** 打开设备USB通信通道
- **hdev:** 设备句柄, 类型为 cando_handle
- **返回值:** True, 成功; False, 失败

2.2.4 cando_close

```
bool cando_close(cando_handle hdev)
```

- **函数说明:** 关闭设备USB通信通道

- **hdev**: 设备句柄, 类型为 `cando_handle`
- **返回值**: True, 成功; False, 失败

2.2.5 cando_get_serial_number_str

```
wchar_t cando_get_serial_number_str(cando_handle hdev)
```

- **函数说明**: 获取设备序列号字符串
- **hdev**: 设备句柄, 类型为 `cando_handle`
- **返回值**: 设备序列号字符串, 类型为 `wchar_t`

2.2.6 cando_get_dev_info

```
bool cando_get_dev_info(cando_handle hdev, uint32_t *fw_version, uint32_t *hw_version)
```

- **函数说明**: 获取设备的固件和硬件版本信息
- **hdev**: 设备句柄, 类型为 `cando_handle`
- ***fw_version**: 存储设备固件版本的变量指针, 如: 32 表示 v3.2
- ***hw_version**: 存储设备硬件版本的变量指针, 如: 12 表示 v3.2
- **返回值**: True, 成功; False, 失败

2.2.7 cando_set_timing

```
bool cando_set_timing(cando_handle hdev, uint16_t ch, cando_bittiming_t *timing)
```

- **函数说明**: 配置CAN波特率相关信息
- **hdev**: 设备句柄, 类型为 `cando_handle`
- **ch**: CAN 通道 0 或 1
- ***timing**: CAN波特率配置结构体指针, 参考[cando_bittiming_t](#)
- **返回值**: True, 成功; False, 失败

2.2.8 cando_start

```
bool cando_start(cando_handle hdev, uint16_t ch, uint32_t mode)
```

- **函数说明**: 启动Cando或Cando_pro
- **hdev**: 设备句柄, 类型为 `cando_handle`
- **ch**: CAN 通道 0 或 1
- **mode**: CAN工作模式, 参考[CANDO_MODE *](#)
- **返回值**: True, 成功; False, 失败

2.2.9 cando_stop

```
bool cando_stop(cando_handle hdev, uint16_t ch)
```

- **函数说明**: 停止Cando或Cando_pro
- **hdev**: 设备句柄, 类型为 `cando_handle`
- **ch**: CAN 通道 0 或 1
- **返回值**: True, 成功; False, 失败

2.2.10 cando_frame_send

```
bool cando_frame_send(cando_handle hdev, cando_frame_t *frame)
```

- **函数说明：**发送CAN数据帧
- **hdev：**设备句柄，类型为 cando_handle
- ***frame：**数据帧结构体指针，参考[cando_frame_t](#)
- **返回值：**True，成功；False，失败

2.2.11 cando_frame_read

```
bool cando_frame_read(cando_handle hdev, cando_frame_t *frame, uint32_t timeout_ms)
```

- **函数说明：**读取CAN数据帧
- **hdev：**设备句柄，类型为 cando_handle
- ***frame：**数据帧结构体指针，参考[cando_frame_t](#)
- **返回值：**True，成功；False，失败

2.3 辅助功能接口

2.3.1 cando_parse_err_frame

```
bool cando_parse_err_frame(cando_frame_t *frame, uint32_t *err_code, uint8_t *err_tx, uint8_t *err_rx)
```

- **函数说明：**解析错误数据帧的错误信息
- ***frame：**错误数据帧结构体指针，参考[cando_frame_t](#)
- ***err_code：**CAN总线错误代码，参考[CAN_ERR *](#)
- ***err_tx：**CAN总线发送错误计数
- ***err_rx：**CAN总线接收错误计数
- **返回值：**True，成功；False，失败